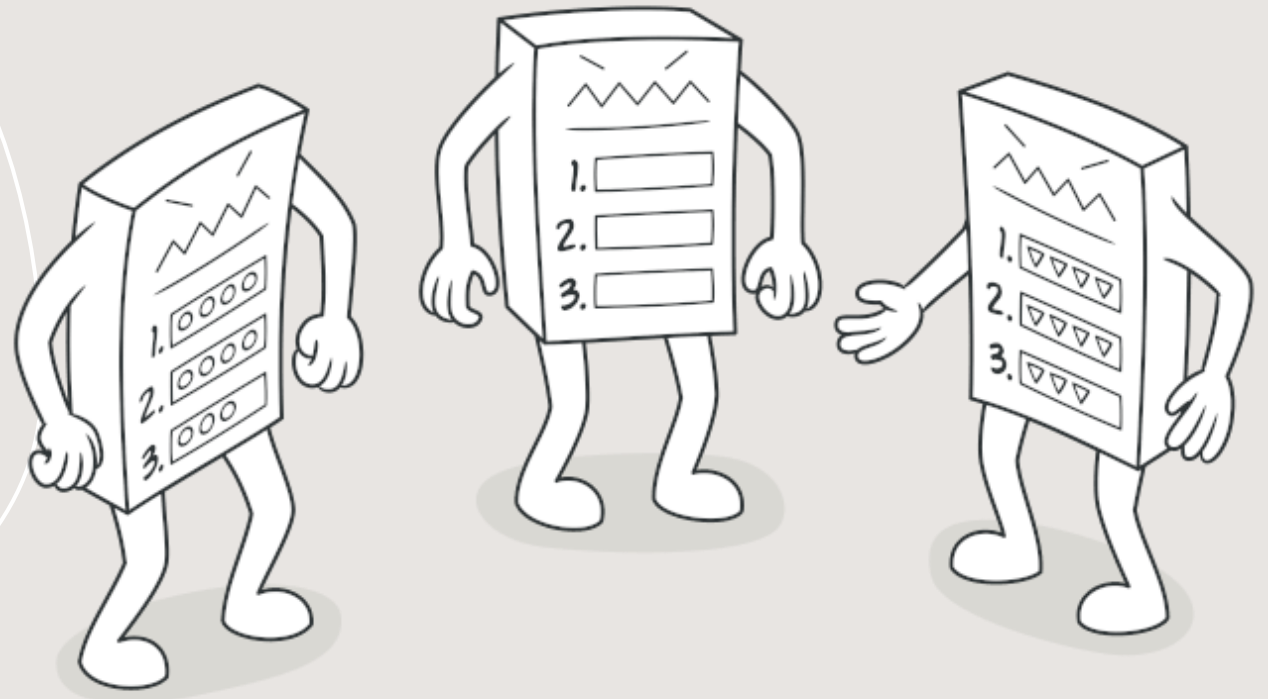


Template

Šablona



Hlavní idea

Template (Šablona) je behaviorální návrhový vzor (vzor související s chováním), který definuje kostru algoritmu v nadtřídě, ale umožňuje podtřídám přepsat konkrétní kroky algoritmu beze změny jeho struktury.



Návrhový vzor Template je s výhodou využívám v situacích, kdy existuje nějaký algoritmus s volitelnými kroky a je vhodné, aby všechny potenciální podtřídy mohly tyto požadované kroky algoritmu implementovat odlišným způsobem, a to bez nutnosti modifikace původního algoritmu



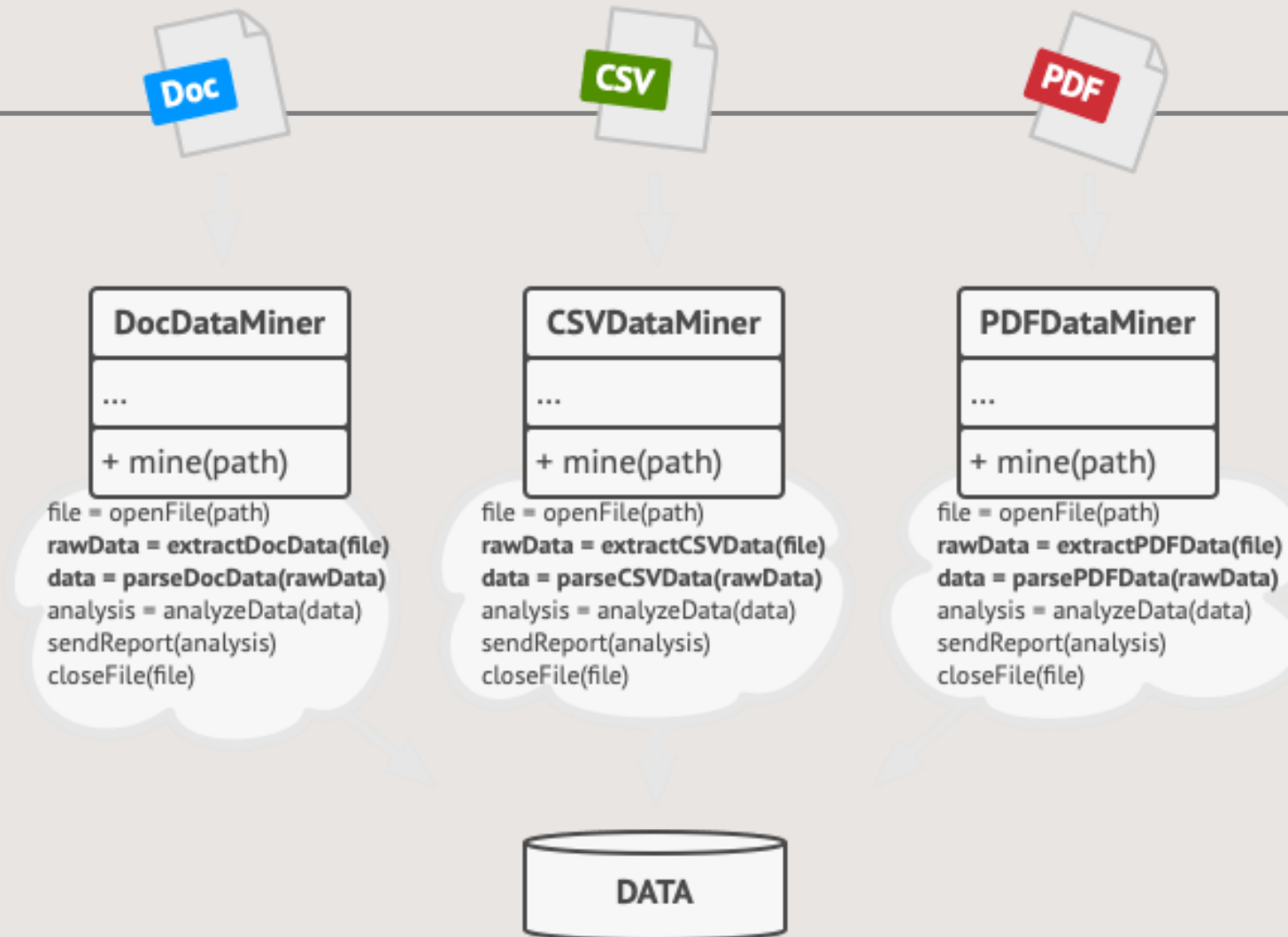
Problém

Představte si, že vytváříte aplikaci pro získávání dat z firemních dokumentů a jejich následnou analýzu. Uživatelé ukládají dokumenty v různých formátech (PDF, DOC, CSV) a rádi by z těchto dokumentů extrahovat smysluplná data v jednotném formátu.

První verze aplikace mohla pracovat pouze se soubory DOC. V následující verzi byla rozšířena možnost o soubory CSV. O měsíc později jste přidali podporu pro data ze souborů PDF.



Problém





Problém

V určitém okamžiku jste si všimli, že všechny tři třídy mají hodně podobný kód. Zatímco kód pro práci s různými formáty dat byl ve všech třídách zcela odlišný, kód pro zpracování a analýzu dat je téměř totožný. Bylo by tedy asi vhodné tuto duplicitu odstranit.

Vyskytl se i další problém související s klientským kódem. V něm je mnoho podmínek, které zvolily správný postup v závislosti na třídě objektu ke zpracování. Pokud by všechny tři třídy zpracování měly společné rozhraní nebo základní třídu, mohli byste eliminovat podmíněnosti v kódu klienta a používat polymorfismus při volání metod na objektu zpracování.



Řešení

Návrhový vzor **Template** navrhuje, abyste algoritmus rozložili na několik kroků, následně přeměnili tyto kroky na metody a vložili sérii volání těchto metod do jedné metody šablony.

Jednotlivé kroky mohou být buď abstraktní, nebo mohou mít nějakou výchozí implementaci. K použití algoritmu se předpokládá, že klient poskytne svou vlastní podtřídu, implementuje všechny abstraktní kroky a v případě potřeby přepíše některé z volitelných (ne však samotnou metodu šablony).



Řešení

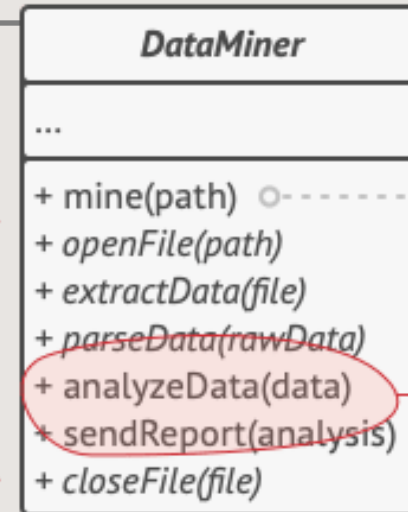
Uvidíme, jak se to projeví v naší aplikaci pro získávání dat. Můžeme vytvořit základní třídu pro všechny tři algoritmy analýzy. Tato třída definuje metodu šablony sestávající ze série volání různých kroků zpracování dokumentů.

Template method

Steps

file = openFile(path)
rawData = extractData(file)
data = parseData(rawData)
analysis = analyzeData(data)
sendReport(analysis)
closeFile(file)

Steps with default implementation.



PDFDataMiner

...
+ openFile(path)
+ extractData(file)
+ parseData(rawData)
+ closeFile(file)

PDF-related overrides.



Řešení

Nejprve můžeme všechny kroky prohlásit za **abstraktní**, čímž donutíme podtřídy, aby poskytovaly své vlastní implementace metod. V našem případě podtřídy již mají všechny potřebné implementace, takže jediná věc, kterou možná budeme muset udělat, je upravit signatury metod tak, aby odpovídaly metodám nadtřídy.

Nyní se podívejme, co můžeme udělat, abychom se zbavili duplicitního kódu. Vypadá to, že kód pro otevírání/zavírání souborů a extrahování/analýzu dat se liší pro různé formáty dat, takže nemá smysl je měnit. Implementace dalších kroků, jako je analýza nezpracovaných dat a vytváření sestav, je však velmi podobná, takže ji lze výjmout do základní třídy, kde mohou podtřídy sdílet tento kód.



Řešení

Jak je vidět, mám několik možných kroků:

- **konkrétní** (společné pro všechny),
- **abstraktní / přepisovatelné** (implementuje potomek)
- případně **hook** (volitelně přepisovatelný krok s default implementací).

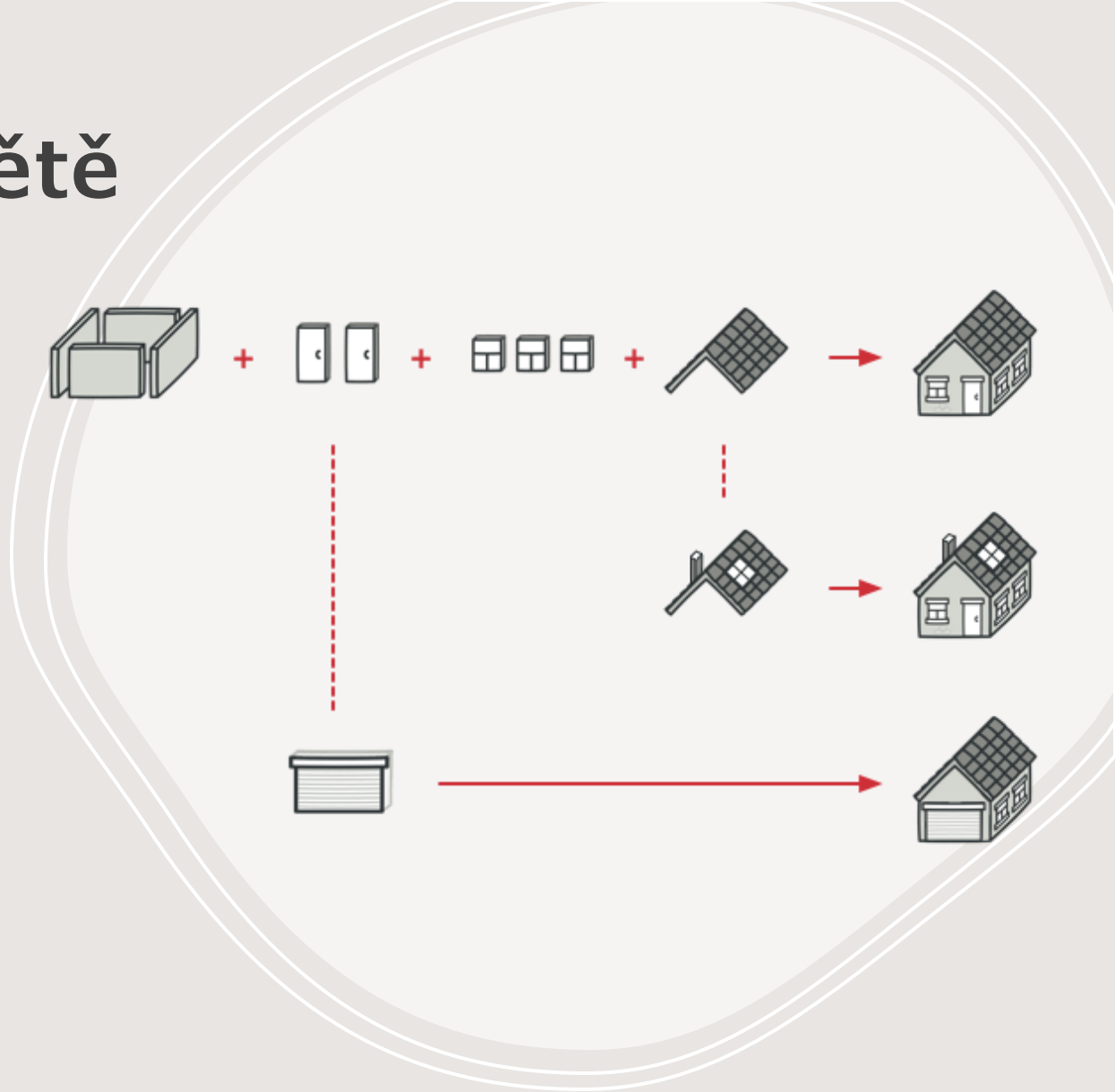
Hooky, nazýváme také háčky. Háček je volitelný krok s prázdným tělem. Metoda šablony by fungovala, i když háček není přepsán. Obvykle jsou háčky umístěny před a za rozhodujícími kroky algoritmů, což poskytuje podtřídám další rozšiřující body pro algoritmus.



Analogie v reálném světě

Architektonický plán pro stavbu standardního domu může obsahovat několik přístaveb, které umožní případnému majiteli upravit některé detaily výsledného domu.

Každý stavební krok, jako je položení základů, stavba zdí, instalace instalatérských a rozvodů vody a elektřiny atd., lze mírně změnit, aby se výsledný dům trochu lišil od ostatních.





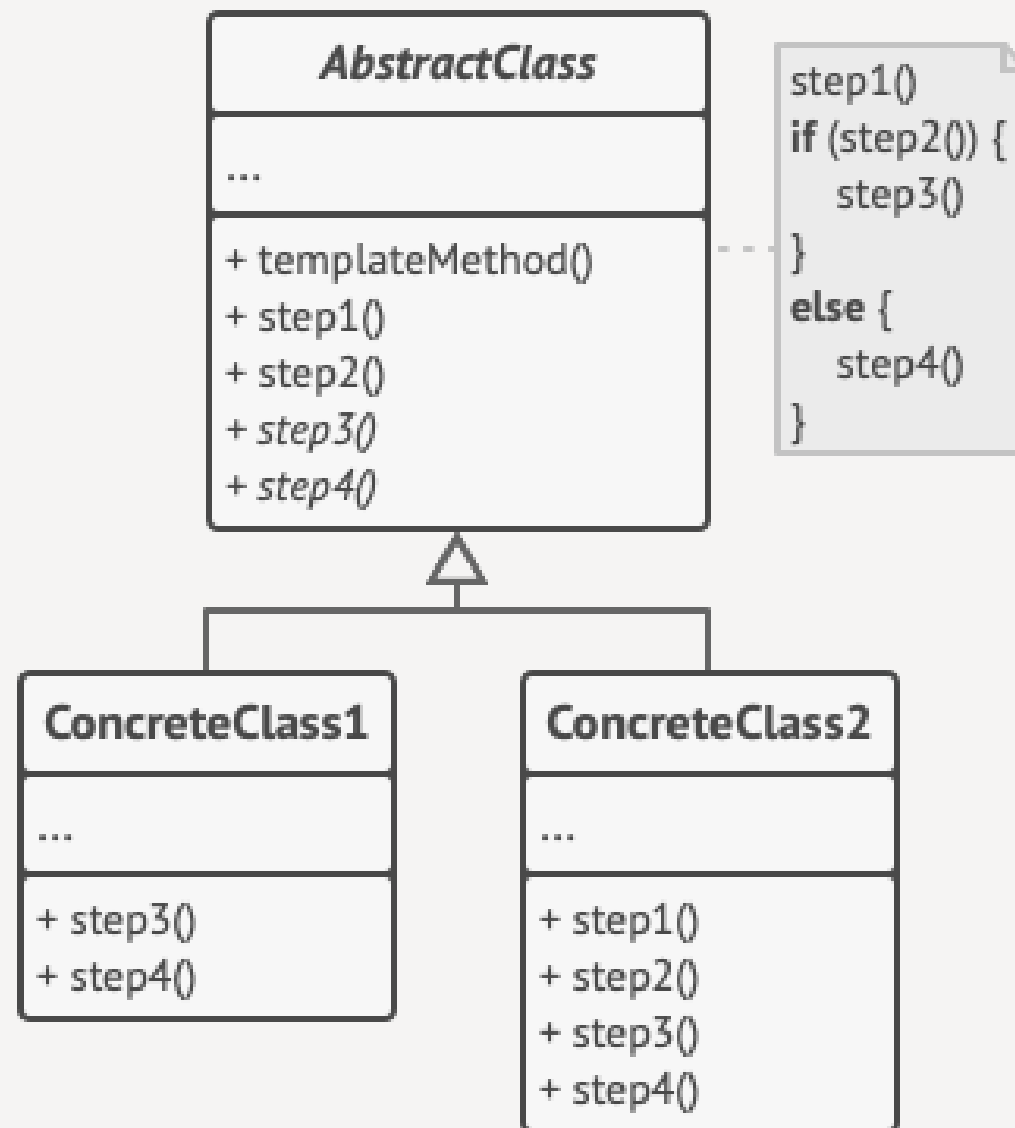
Výhody a nevýhody



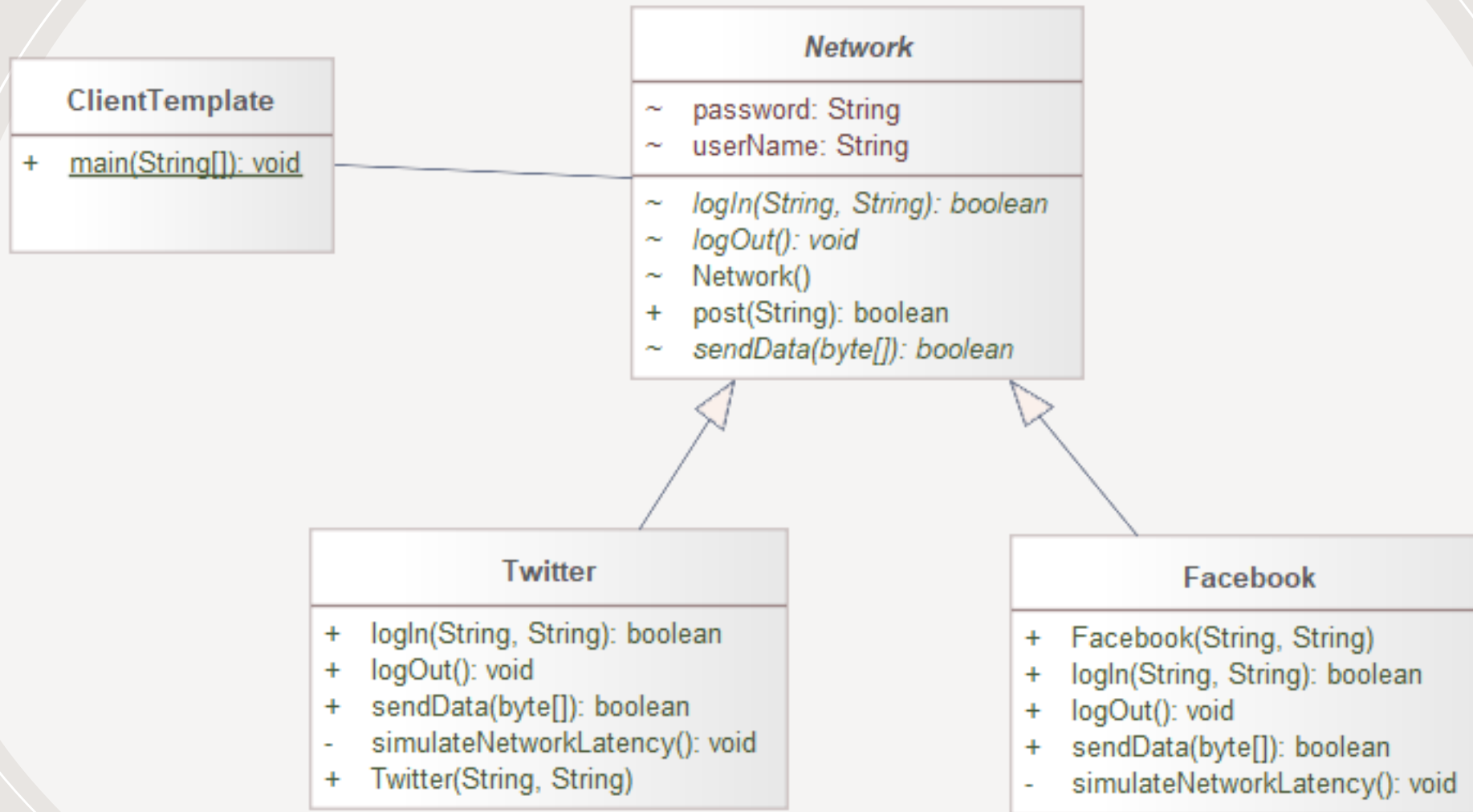
- Klientům můžete dovolit, aby přepsali pouze určité části velkého algoritmu, takže je méně ovlivní změny, ke kterým dochází v jiných částech algoritmu.
- Duplicitní kód můžete stáhnout do nadtřídy



- Někteří klienti mohou být omezeni poskytnutou kostrou algoritmu.
- Metody šablony mohou být obtížně udržitelné, pokud mají mnoho kroků.



Struktura



Implementace



```
public abstract class Network {
    String userName;
    String password;
    Network() {}

    public boolean post(String message) {
        if (logIn(this.userName, this.password)) {
            // Send the post data.
            boolean result = sendData(message.getBytes());
            logOut();
            return result;
        }
        return false;
    }

    abstract boolean logIn(String userName, String password);
    abstract boolean sendData(byte[] data);
    abstract void logOut();
}
```



```
public class Facebook extends Network {  
    public Facebook(String userName, String password) {  
        this.userName = userName;  
        this.password = password;  
    }  
  
    public boolean login(String userName, String password) {  
        System.out.println("\nChecking user's parameters");  
        System.out.println("Name: " + this.userName);  
        System.out.print("Password: ");  
        for (int i = 0; i < this.password.length(); i++) {  
            System.out.print("*");  
        }  
        simulateNetworkLatency();  
        System.out.println("\n\nLogIn success on Facebook");  
        return true;  
    }  
  
    public boolean sendData(byte[] data) {  
        boolean messagePosted = true;  
        if (messagePosted) {  
            System.out.println("Message: '" + new String(data)  
                + "' was posted on Facebook");  
            return true;  
        } else {  
            return false;  
        }  
    }  
  
    public void logout() {  
        System.out.println("User: '" + userName  
            + "' was logged out from Facebook");  
    }  
  
    private void simulateNetworkLatency() {  
        try {  
            int i = 0;  
            System.out.println();  
            while (i < 10) {  
                System.out.print(".");  
                Thread.sleep( millis: 500);  
                i++;  
            }  
        } catch (InterruptedException ex) {  
            ex.printStackTrace();  
        }  
    }  
}
```

```
public class Twitter extends Network {  
    public Twitter(String userName, String password) {  
        this.userName = userName;  
        this.password = password;  
    }  
  
    public boolean login(String userName, String password) {  
        System.out.println("\nChecking user's parameters");  
        System.out.println("Name: " + this.userName);  
        System.out.print("Password: ");  
        for (int i = 0; i < this.password.length(); i++) {  
            System.out.print("*");  
        }  
        simulateNetworkLatency();  
        System.out.println("\n\nLogIn success on Twitter");  
        return true;  
    }  
  
    public boolean sendData(byte[] data) {  
        boolean messagePosted = true;  
        if (messagePosted) {  
            System.out.println("Message: '" + new String(data)  
                + "' was posted on Twitter");  
            return true;  
        } else {  
            return false;  
        }  
    }  
  
    public void logout() {  
        System.out.println("User: '" + userName  
            + "' was logged out from Twitter");  
    }  
  
    private void simulateNetworkLatency() {  
        try {  
            int i = 0;  
            System.out.println();  
            while (i < 10) {  
                System.out.print(".");  
                Thread.sleep( millis: 500);  
                i++;  
            }  
        } catch (InterruptedException ex) {  
            ex.printStackTrace();  
        }  
    }  
}
```



```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
```

```
public class ClientTemplate {
```

```
    public static void main(String[] args) throws IOException {
        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
        Network network = null;
        System.out.print("Input user name: ");
        String userName = reader.readLine();
        System.out.print("Input password: ");
        String password = reader.readLine();

        // Enter the message.
        System.out.print("Input message: ");
        String message = reader.readLine();

        System.out.println("\nChoose social network for posting message.\n" +
            "1 - Facebook\n" +
            "2 - Twitter");
        int choice = Integer.parseInt(reader.readLine());

        // Create proper network object and send the message.
        if (choice == 1) {
            network = new Facebook(userName, password);
        } else if (choice == 2) {
            network = new Twitter(userName, password);
        }
        network.post(message);
    }
}
```



Zdroje

<https://refactoring.guru/design-patterns>

<http://voho.eu/wiki/navrhovy-vzor/>

<https://www.oodeesign.com/>